A Universal Partial Cycle with Three Wildcards per Window

ZIJIAN ZENG, UCSI University, Malaysia

A universal partial cycle covers every word of a fixed length exactly once by cyclic windows, allowing a wildcard to match any alphabet symbol. The number of wildcards in each window is its diamondicity. Fillmore, Goeckner, Kirsch, Martin and McGinnis asked whether a nontrivial universal partial cycle of diamondicity greater than one exists. We answer this question affirmatively by giving a binary cycle of length 512 whose length-12 windows each contain three wildcards. Its 512 windows partition the 4096 binary words into sets of eight completions.

The complete cycle is specified by 128 octal digits. A short, solver-free Python program checks every pair of cyclic windows and every binary 12-word, giving a reproducible finite existence proof. The cycle also realizes the regime in which the window length does not divide the cycle length.

Zijian Zeng, UCSI University.
1002266693@ucsiuniversity.edu.my.

1 The existence question

A De Bruijn cycle represents every word of a fixed length by an overlapping cyclic window. Chen, Kitaev, Mütze and Sun introduced universal partial words and their cyclic versions, which compress this representation by allowing wildcards [CKMS17]. Write $\diamond$ for a wildcard, matching either binary symbol independently at each occurrence. For a cyclic word $u = u_0 \cdots u_{L-1}$ over $\{0, 1, \diamond\}$, its window at start i is

$$W_i = (u_i, u_{i+1}, \ldots, u_{i+n-1}), \qquad 0 \le i < L,$$

where subscripts are reduced modulo L. A total word $x \in \{0,1\}^n$ is *compatible* with W_i when $x_k = u_{i+k}$ at every offset k where the latter character is ordinary. The word u is a universal partial cycle, or *upcycle*, if every x has exactly one compatible starting index. Starting indices remain distinct even when windows overlap. The cycle is nontrivial when it contains both an ordinary symbol and a wildcard. If each window has d wildcards, its *diamondicity* is d.

Fillmore, Goeckner, Kirsch, Martin and McGinnis give binary examples with $n = 8$ and $d = 1$ in Theorem 3.1 of [FGK$^+$26]. Their Question 9.2 asks whether upcycles with $d > 1$ exist. We prove the following.

Theorem 1.1. *There is a nontrivial binary upcycle for words of length* 12 *with diamondicity* 3 *and cyclic length* 512.

The example establishes that multiple independent wildcards in every window can coexist with unique coverage in a single cycle. The exact object and all verification code appear below.

The closest prior constructions in [FGK$^+$26] supply the $d = 1$ seeds and multiply alphabet size while preserving n and d (Theorem 5.1 and Corollary 5.2). The present $d = 3$ seed extends the known range of diamondicities. Its wildcard positions have period four. This meets the necessary condition $d = n/4$ in their Corollary 8.14 for a nontrivial period-four *frame*, the pattern recording which positions are wildcards.

Carey, Kearney, Kirsch and Popescu study universal partial quasi-families in Definition 8.2 of [CKKP25]. Such a family may use several cyclic words to cover all total words. Here the entire cover is realized by one cyclic word, including the windows crossing its boundary.

2 The exact cycle and its proof

Concatenate the following four rows, in order, to form a string P of 128 octal digits. Row breaks are purely typographical.

```
00177617563754173657321716771457
12371016661655164636441633162656
22161556133611160676045602151214
46141314063206120412261006540236
```

For each digit b, let $\beta(b)$ be its three-bit binary expansion, including leading zeroes. Replace b by $\diamond\beta(b)$ and concatenate the resulting blocks to obtain u. For example, the digits $0, 1, 7$ decode as $\diamond000, \diamond001, \diamond111$. The last block is immediately followed by the first block when u is read cyclically. In the code below, the ASCII character `*` represents $\diamond$.

For a window W_i, let $C_i \subseteq \{0,1\}^{12}$ be its set of compatible total words. The following elementary criterion reduces unique coverage to finite checks on ordinary symbols.

Lemma 2.1. *Two partial words of the same length have disjoint sets of binary completions if and only if at some common offset both characters are ordinary and unequal.*

Proof. An unequal pair of specified bits prevents any simultaneous completion. Conversely, suppose there is no such pair. At each offset where either word specifies a bit, choose that bit; the choice is consistent whenever both specify one. Choose either bit at any remaining offset. This gives a common completion. □

Proof of Theorem 1.1. The decoding gives $128 \cdot 4 = 512$ positions, with wildcards exactly at indices congruent to zero modulo four. Since four divides 512, this description is valid across the cyclic boundary. Every 12-position window therefore contains three wildcards and nine ordinary symbols. The cycle is nontrivial, and its length exceeds 12; in particular, no physical position occurs twice within a window. Each C_i has exactly $2^3 = 8$ members.

For the displayed word, the finite check in the first nested loop of Listing 1 verifies

$$\forall\, 0 \le i < j < 512 \quad \exists\, 0 \le k < 12 : \quad u_{i+k}, u_{j+k} \in \{0,1\}, \quad u_{i+k} \ne u_{j+k}. \tag{1}$$

It checks all $\binom{512}{2} = 130816$ pairs, using the characters of the same decoded word and cyclic subscripts. Every test succeeds. By Lemma 2.1, the sets C_i are pairwise disjoint. Their union, a subset of $\{0,1\}^{12}$, has cardinality

$$\sum_{i=0}^{511} |C_i| = 512 \cdot 8 = 4096 = 2^{12}.$$

Thus these sets partition all binary 12-words. Each total word is compatible with exactly one starting index, proving the theorem. □

This is a computer-assisted finite proof: the pair check is an exhaustive integer and character computation, and its full input and implementation are printed here. The second loop of the same listing directly checks the conclusion by comparing each of the 4096 total words with all 512 windows. Both checks use only the displayed word and standard Python operations.

3 Complete reproducible verification

Listing 1 is a complete Python 3 program using only the standard library. Run it normally, with assertions enabled, for example as `python3 verify.py`; do not use the `-O` option. It constructs every window with explicit reduction modulo 512, so the eleven starts $501, \ldots, 511$ that wrap around are included. It then checks both pairwise incompatibility and unique coverage, without sampling.

Listing 1. Complete solver-free verification of the displayed cycle.

```
"""Exhaustive check of the printed binary upcycle; run without -O."""
from itertools import product

P = (
    '00177617563754173657321716771457'
    '12371016661655164636441633162656'
    '22161556133611160676045602151214'
    '46141314063206120412261006540236'
)
u = ''.join('*' + format(int(b), '03b') for b in P)
L, n, d = len(u), 12, 3
assert len(P) == 128 and L == 512 == 2 ** (n - d)
assert L > n and set(u) == {'*', '0', '1'}
assert all((c == '*') == (i % 4 == 0)
           for i, c in enumerate(u))
W = [''.join(u[(i + k) % L] for k in range(n))
     for i in range(L)]
assert all(w.count('*') == d for w in W)

pairs = 0
for i in range(L):
    for j in range(i + 1, L):
        assert any(a != '*' and b != '*' and a != b
                   for a, b in zip(W[i], W[j]))
        pairs += 1
assert pairs == 130816

counts = []
for x in product('01', repeat=n):
    count = sum(all(c == '*' or c == b
                    for c, b in zip(w, x)) for w in W)
    counts.append(count)
assert len(counts) == 4096 and min(counts) == max(counts) == 1
print('Verified: 512 windows, 130816 pairs, 4096 words.')
```

Successful execution prints

```
Verified: 512 windows, 130816 pairs, 4096 words.
```

The direct coverage loop performs $4096 \cdot 512 = 2097152$ window compatibility tests. In particular, it counts starting indices rather than different window strings or rotation classes. Overlapping windows read the same characters of u; no mismatch predicate is treated as an independent variable. All counts in this program are exact integers.

4 Discovery and further questions

The witness was obtained by an AI-assisted SAT search using exact coverage of binary 12-words by partial 12-tiles, balance at literal 11-character overlaps, and guarded cuts to join selected edges into one component. An Euler circuit then supplied the cyclic word. The finite proof above verifies the resulting object directly. Separate implementations checked ordinary conflicts and exhaustive coverage, using published $d = 1$ examples as positive controls and rejecting bit-damaged and all-zero examples.

The theorem supplies an affirmative answer to Question 9.2 of [FGK+26]. It also realizes the nondivisibility regime raised in the paragraph immediately following that question: $12 \nmid 512 = 2^{12-3}$. The next questions concern the possible parameters: does diamondicity 2 occur, what is the smallest order supporting diamondicity greater than one, and can a uniform construction produce unbounded diamondicity?

AI methodology

OpenAI Codex provided substantial assistance with the search formulation and implementation, witness discovery, verification programs, literature comparison, proof exposition and manuscript preparation. This assistance materially affected all sections and the code.

References

[CKKP25] William D. Carey, Matthew David Kearney, Rachel Kirsch, and Stefan Popescu. Universal partial tori. *Designs, Codes and Cryptography*, 93:2803–2822, 2025. https://doi.org/10.1007/s10623-025-01609-9.

[CKMS17] Herman Z. Q. Chen, Sergey Kitaev, Torsten Mütze, and Brian Y. Sun. On universal partial words. *Discrete Mathematics & Theoretical Computer Science*, 19(1), 2017. Paper 16. https://doi.org/10.23638/DMTCS-19-1-16.

[FGK+26] Dylan Fillmore, Bennet Goeckner, Rachel Kirsch, Kirin Martin, and Daniel McGinnis. The existence and structure of universal partial cycles. *Designs, Codes and Cryptography*, 94, 2026. Article 152. https://doi.org/10.1007/s10623-026-01841-x.